



COMPREHENSIVE PRACTICE EXERCISE

Program specification

You are asked to develop a Windows Forms application to manage the event bookings of a small function venue (see **Figure 1**). The venue hosts weddings and conferences. All bookings share similar characteristics, such as the client's name, the event date, the number of guests, and the event duration in hours. However, some types of bookings have additional information. For example, weddings include the theme chosen by the couple, whereas conferences are held on behalf of a specific organisation. Every booking also receives a quote, which is calculated using the venue's rate card (see **Table 1**). The system should allow the user to add and manage different types of bookings and their corresponding details (see **Figure 2**, **Figure 3**, and **Figure 4**), save new bookings to the venue's existing booking file (see **Figure 5** and **Figure 6**) and view the list of bookings (see **Figure 7** and **Figure 8**).

1. Preparation

1.1. Download the provided files:

1.1.1. Download the zipped folder named **VenueBookings_Provided.zip**.

1.2. View provided files:

1.2.1. The following has been provided:

- Complete main Windows Forms (frmVenueBookings) design
- Complete dialog form (dlgAddBooking) design.
- Partial BookingManager static class.
- Partial back-end code for both forms.
- Partial Booking class.
- .cs files for derived classes and enum

1.2.2. Open the solution file (.sln file) and consider the provided Windows Forms application. Unless stated otherwise, **do not** change the main Windows Form design, the dialog form design, or any provided code. You may add **using** directives as necessary.

2. Developing your program

2.1. EventType

- 2.1.1. Add a public enum, EventType, in a separate file that will allow a user to specify whether an event is a Conference or Wedding.

2.2. Complete BookingManager

- 2.2.1. Declare a private static two-dimensional array of type decimal named ratesPerGuest. Initialise the array with the venue's rate card as shown in **Table 1**. Each row of the array represents the number of guests as a range, and each column represents the event duration.

Table 1 - Rate card (rate per guest)

Guests \ Duration	1 – 4 hours	5 – 8 hours	More than 8 hours
1 – 50	R320	R450	R560
51 – 150	R280	R390	R490
More than 150	R240	R340	R430

2.3. Booking

- 2.3.1. Booking should only be used for inheritance. It should not be possible to instantiate objects of the class.
- 2.3.2. Add an auto-implemented property, TypeOfEvent, to allow the user to specify the event type based on the values of the EventType enum. Ensure that only the base class and its derived classes can set the value of this property.
- 2.3.3. Complete the provided constructor to accept the details of a booking and assign the values to the relevant properties. Ensure that the constructor can only be called by the derived classes.
- 2.3.4. Develop a method, PrepareFileRecord, that does not accept anything and returns a string. All derived classes **must** be forced to implement this method.
- 2.3.5. Develop a method, CalculateQuote, that does not accept anything and returns a decimal. The method should calculate the standard quote for a booking by multiplying the number of guests by the per-guest rate. Use an appropriate method in BookingManager to obtain the rate per guest. Derived classes should have the **option** to change how a quote is calculated.

2.4. Derived classes (WeddingBooking and ConferenceBooking)

- 2.4.1. Update the class definitions of each class to relate them to Booking through inheritance.
- 2.4.2. Implement properties specific to each of the derived classes. A wedding booking should have a string property named Theme. A conference booking should have a string

property named `Organisation`. It should not be possible to assign values to either property from outside the class.

- 2.4.3. Both derived classes should have only one constructor that accepts all of the booking details, i.e., the shared details and class-specific details. The shared booking details must be sent to `Booking` to initialise the necessary inherited properties. Additionally, initialise the `TypeOfEvent` property, as well as any properties specific to each derived class, in the derived class's constructor.
- 2.4.4. Each derived class should override the `PrepareFileRecord` method. This method must build and return a string containing a booking's details, prepared in the appropriate CSV format. Implement the method for each class as follows:
 - 2.4.4.1. **Wedding Booking:** Generate a string with the wedding booking's details (properties) separated by a delimiter (either a semi-colon or a comma, depending on your PC's settings). See **Figure 9** for an example of the string format.
 - 2.4.4.2. **Conference Booking:** Generate a string with the conference booking's details (properties) separated by a delimiter (either a semi-colon or comma, depending on your PC's settings). See **Figure 9** for an example of the string format.
- 2.4.5. `WeddingBooking` should override the `CalculateQuote` method. The quote for a wedding is the standard quote plus a once-off décor fee of R8 500. Declare the décor fee as a constant in the class. You **must** call the base class's implementation of the method to obtain the standard quote. `ConferenceBooking` should not override `CalculateQuote`, as conferences are charged the standard quote.

2.5. Complete dialog form

- 2.5.1. Add a public property named **`NewBooking`** of type `Booking`. It should not be possible to assign a value to this property outside of the class.
- 2.5.2. Complete the `btnOK_Click` event handler. Determine whether the user has chosen to add a wedding or a conference. Depending on the user's selection, instantiate an object of the appropriate derived class using the `Booking` property created in **Question 2.5.1**. Assume that you do not have to implement any defensive programming for numerical values. Ensure that the button will return the appropriate `DialogResult` value.

2.6. Complete frmVenueBookings

- 2.6.1. Complete the `btnAddABooking_Click` event handler. Instantiate an object of the dialog form class using a `using` statement. Display the dialog form. If the dialog form returns a `DialogResult` of `OK` and if the form's `Booking` property is not null, add the booking to the list of bookings in `BookingManager`. Then, call the provided

AddBookingToListView method to display the booking and its quote in the ListView. See **Figure 2**, **Figure 3**, and **Figure 4** for the effect of this event handler.

- 2.6.2. Complete the btnSaveList_Click event handler to allow the user to save a booking or multiple bookings to a CSV file (see **Figure 5** and **Figure 6**). Allow the user to select a CSV file using the provided SaveFileDialog control, **d1gSave**. Open a connection with write access to the selected CSV file. Any changes should be appended to the specified file. Open a StreamWriter. Then, using a foreach loop, iterate through the list of bookings in BookingManager and use an appropriate method available to bookings to write the record of a booking's details to the CSV file. Ensure that each record will be written on a new line. Display a confirmation message as in **Figure 6**.
- 2.6.3. Complete the btnViewBookingList_Click event handler to allow the user to load the list of bookings from a CSV file (see **Figure 7** and **Figure 8**). Clear the ListView items. Allow the user to select a CSV file using the provided OpenFileDialog control, **d1gOpen**. Open a connection with read access to the selected file. Then use a StreamReader to read the file contents as outlined below until you reach the end of the file. Create a StreamReader using this file stream. Read each line and split the data using an appropriate delimiter. Use the array returned to display the booking details in the ListView. Ensure that you do not display the CSV file header in the ListView.

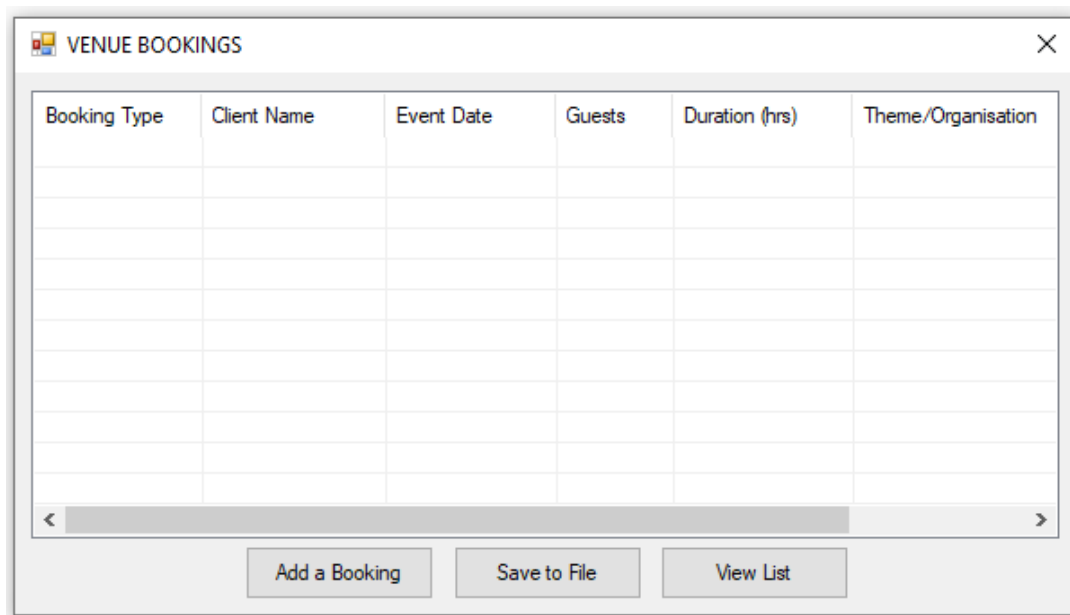
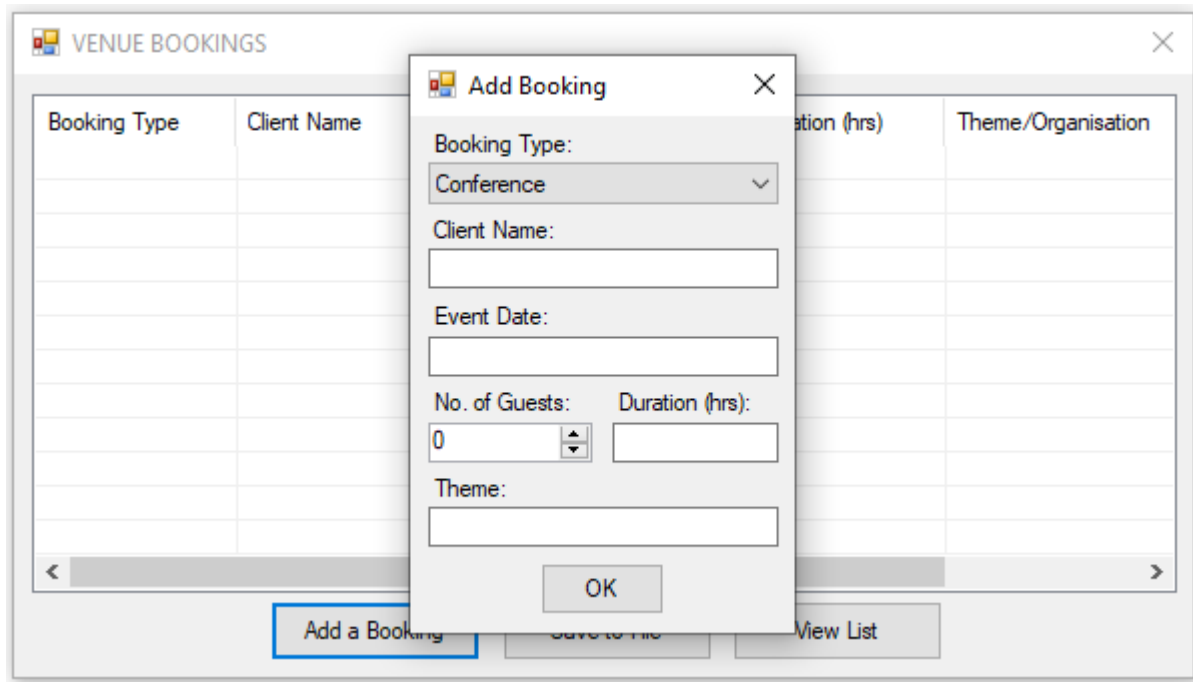


Figure 1: Initial Screen.



**Figure 2: The user would like to add a new booking and clicks on “Add a Booking”.
A dialog form is displayed.**

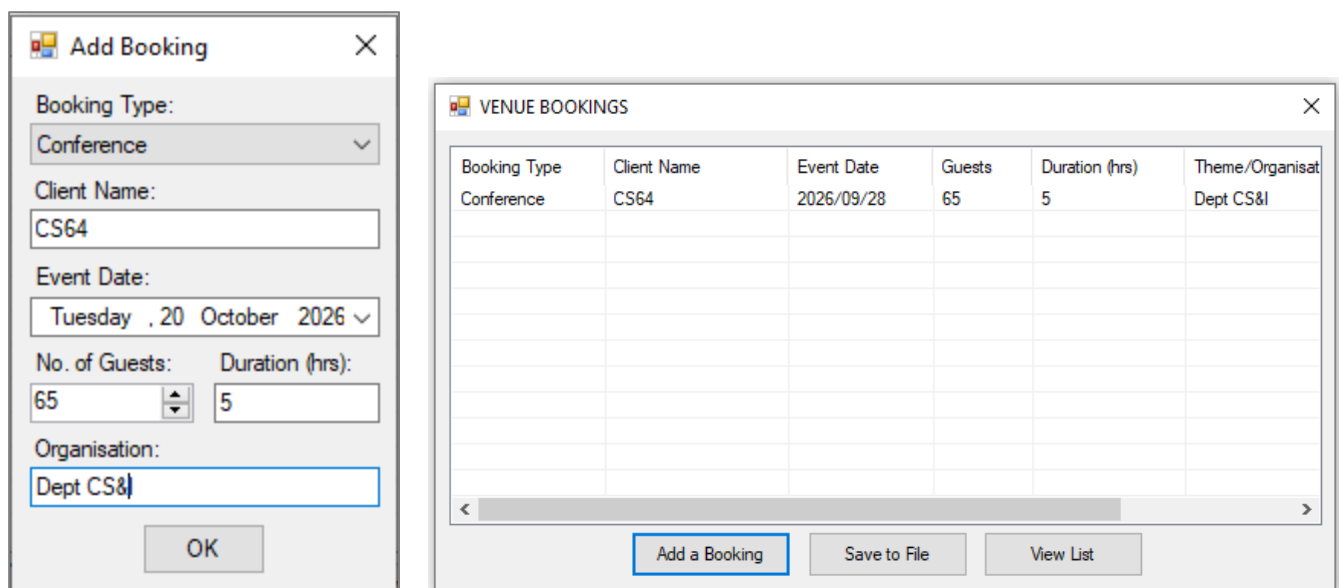


Figure 3: The user opts to add a conference booking. The user provides the necessary booking details and clicks “OK”. The new booking is added to the list of bookings and then displayed in the ListView.

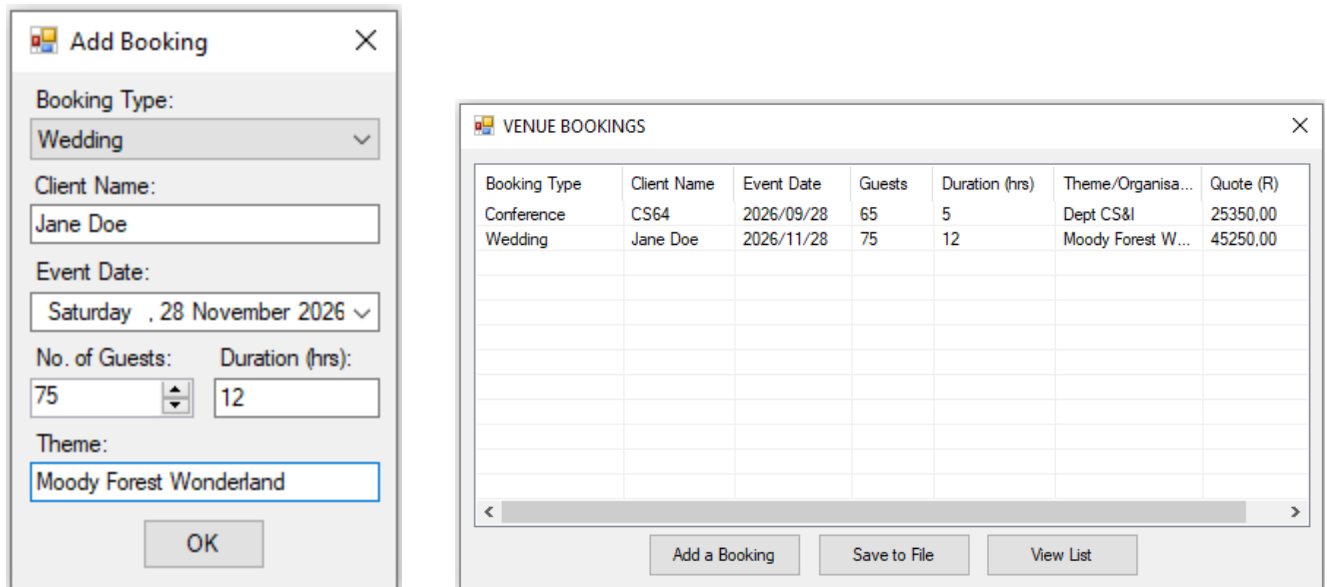


Figure 4: The user opts to add a wedding booking. The user provides the necessary booking details and clicks “OK”. The new booking is added to the list of bookings and then displayed in the ListView. Note the quotation prices displayed.

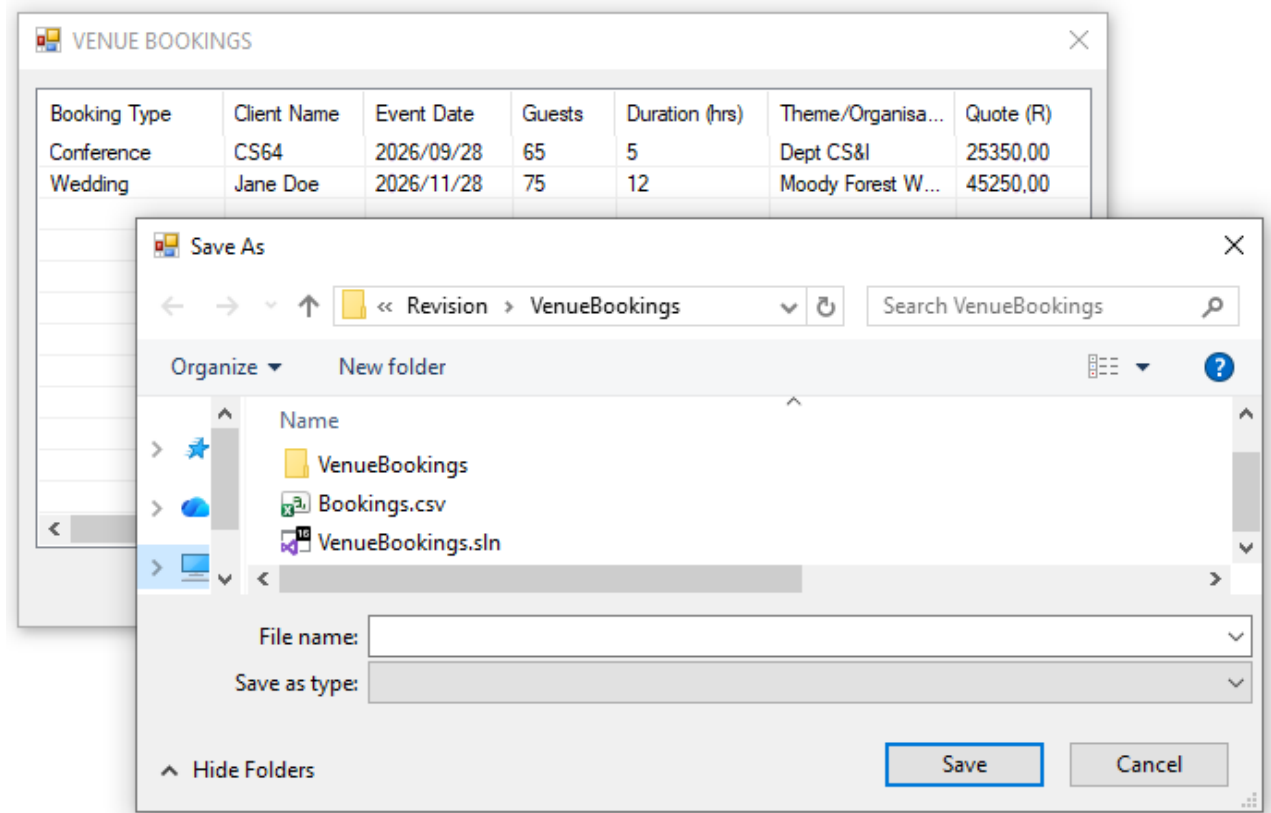


Figure 5: The venue facility would like to save the new bookings. The user clicks “Save to File”. A dialog box is displayed, allowing the user to select a file to append the booking information to and save the updated list.

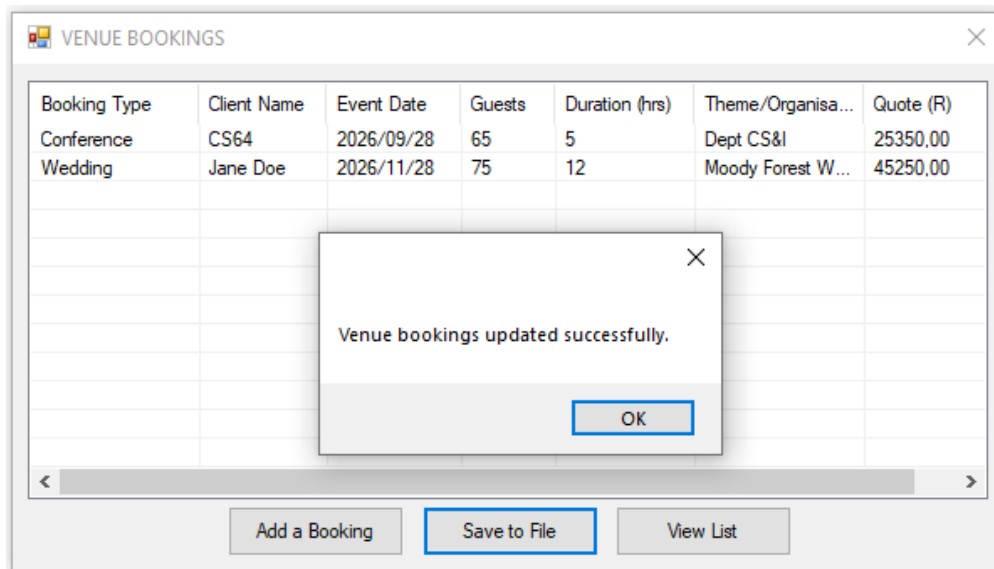


Figure 6: The venue's bookings list is successfully updated, and a confirmation message is displayed.

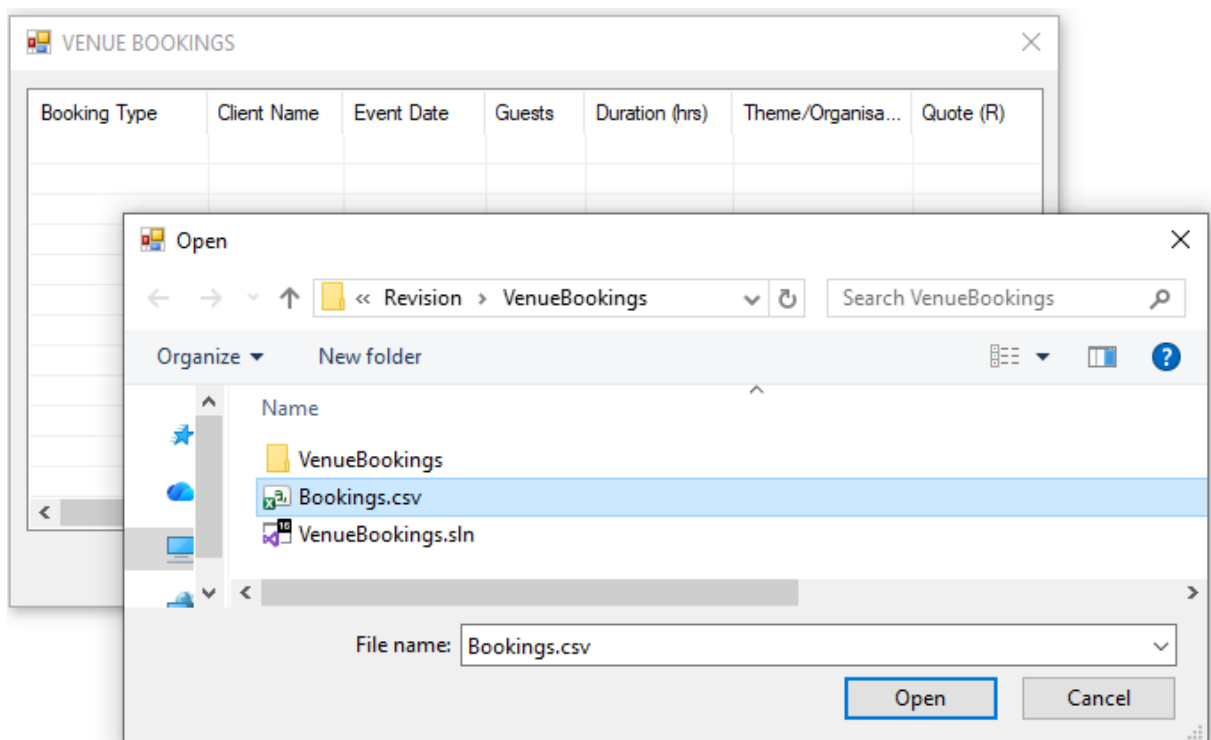


Figure 7: A staff member of the venue facility would like to view the list of bookings and clicks "View List". The ListView is cleared, and a dialog box is displayed to allow the user to select a file. The user selects a CSV file.

Booking Type	Client Name	Event Date	Guests	Duration (hrs)	Theme/Organisa...	Quote (R)
Wedding	Thandi Mo...	2026-12-12	120	6	Rustic Garden	55300.00
Conference	Anele Nkosi	2026-10-20	60	3	Mangaung Tech...	16800.00
Conference	CS64	2026/09/28...	65	5	Dept CS&I	25350,00
Wedding	Jane Doe	2026/11/28...	75	12	Moody Forest W...	45250,00

Buttons: Add a Booking, Save to File, View List

Figure 8: The venue’s entire bookings list is displayed in the ListView.

```

Wedding;Thandi Mokoena;2026-12-12;120;6;Rustic Garden;55300.00
Conference;Anele Nkosi;2026-10-20;60;3;Mangaung Tech Hub;16800.00

```

Figure 9: Example of a string with a booking’s details separated by a delimiter (semi-colon).